シーケンス図・処理フロー

Loggit

Version 1.0　　作成日：2026 年 2 月 23 日

1. レイヤー凡例

レイヤー	役割
Client	React フロントエンド（ブラウザ）
Router	Laravel ルーター（routes/api.php）
Middleware	認証チェック（Sanctum）・Policy 権限チェック
Controller	リクエスト受付・FormRequest バリデーション・レスポンス返却
Service	ビジネスロジック・トランザクション管理
Model	Eloquent モデル・DB 操作
DB	MySQL（FK 制約・CHECK 制約・インデックス）
Scheduler	Laravel スケジューラ（cron）

2. 認証フロー

2.1 ユーザー登録

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	POST /api/register に name・email・password を送信
2	Controller	RegisterRequest でバリデーション（必須・email 形式・min:8・UNIQUE 確認）
3	Service	password をハッシュ化（bcrypt）して User::create()
4	DB	users テーブルに INSERT。user_settings レコードも自動作成
5	Service	Sanctum の createToken()でアクセストークン発行
6	Controller	token + user を JSON で返却（201）

例外系

▶️ **email が既に存在する：** 422 / errors.email: ['このメールアドレスはすでに使用されています']

2.2 ログイン

Client	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	POST /api/login に email・password を送信
2	Controller	LoginRequest でバリデーション（必須チェック）
3	Service	Auth::attempt()で認証。失敗時は ThrottleException 確認
4	DB	users テーブルから email 検索・bcrypt 照合
5	Service	認証成功：既存トークンを削除して createToken()で新規発行
6	Controller	token + user を JSON で返却（200）

例外系

▶️ **認証失敗：** 422 / message: 'メールアドレスまたはパスワードが正しくありません'

3. プロジェクトフロー

3.1 プロジェクト作成

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	POST /api/projects に name・color・due_date 等を送信
2	Middleware	Sanctum 認証チェック（トークン検証）
3	Controller	StoreProjectRequest でバリデーション
4	Service	DB::transaction()開始
5	Model	Project::create()でプロジェクト作成
6	Model	ProjectStatus::insert()でデフォルトステータス 3 件を自動生成（Todo/In Progress/Done）
7	DB	projects と project_statuses に INSERT
8	Controller	ProjectResource でレスポンス整形（ステータス含む）して返却（201）

3.2 ステータス名カスタマイズ

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	PUT /api/projects/{id}/statuses/{statusId} に name を送信
2	Middleware	Sanctum 認証 → ProjectPolicy で所有権チェック
3	Controller	UpdateStatusRequest でバリデーション（必須・max:100）
4	Model	ProjectStatus::find()→update(['name' => $name])
5	Controller	更新後のステータスを JSON で返却（200）

例外系

- ▶️ **他ユーザーのプロジェクット：** 403 Forbidden
- ▶️ **存在しないステータス ID：** 404 Not Found

4. タスクフロー

4.1 タスク作成（サブタスク含む）

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	POST /api/projects/{id}/tasks に title・status_id・parent_task_id 等を送信
2	Middleware	Sanctum 認証 → TaskPolicy で所有権チェック
3	Controller	StoreTaskRequest でバリデーション
4	Service	parent_task_id が指定された場合、TaskService::getDepth()で階層数を計算
5	Service	depth >= 3 なら TaskHierarchyException をスロー → 422
6	Service	循環参照チェック（parent_task_id に自分の id が含まれないか確認）
7	Model	Task::create()でタスク作成
8	DB	tasks テーブルに INSERT
9	Controller	TaskResource でレスポンス整形して返却（201）

例外系

- ▶️ **3 階層超過：** 422 / message: 'サブタスクは 3 階層までしか作成できません'

▶️ **循環参照：** 422 / message: '循環参照となるサブタスクは作成できません'
▶️ **他ユーザーのプロジェクト：** 403 Forbidden

4.2 タスク削除（ソフトデリート）と復元

Client	Middleware	Controller	Model	DB

削除

Step	レイヤー	処理内容
1	Client	DELETE /api/tasks/{id} を送信
2	Middleware	Sanctum 認証 → TaskPolicy で所有権チェック
3	Model	Task::find()->delete()（SoftDeletes トレイト）
4	DB	deleted_at に現在時刻をセット。サブタスクも再帰的にソフトデリート
5	Controller	204 No Content を返却

復元

Step	レイヤー	処理内容
1	Client	POST /api/tasks/{id}/restore を送信
2	Middleware	Sanctum 認証 → TaskPolicy で所有権チェック（withTrashed()で取得）
3	Model	Task::withTrashed()->find()->restore()
4	DB	deleted_at を NULL に更新。サブタスクも再帰的に復元
5	Controller	復元後のタスクを JSON で返却（200）

5. 活動ログ メインフロー

5.1 計測開始

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	POST /api/activity-logs/start に project_id・note を送信
2	Middleware	Sanctum 認証チェック
3	Controller	StartActivityLogRequest でバリデーション

Step	レイヤー	処理内容
4	Service	DB::transaction()開始
5	Service	ActivityLog::where('user_id')->whereNull('ended_at')->lockForUpdate()->first()
6	Service	計測中ログが存在する → AlreadyTrackingException → 422
7	Service	ActivityLog::create(['user_id', 'project_id', 'started_at' => now(), 'note'])
8	DB	activity_logs に INSERT（started_at はサーバー時刻）
9	Controller	作成したログを ActivityLogResource で返却（201）

例外系

- ▶️ **すでに計測中のログあり：** 422 / message: 'すでに計測中の活動ログがあります'

5.2 計測停止

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	POST /api/activity-logs/{id}/stop を送信
2	Middleware	Sanctum 認証 → ActivityLogPolicy で所有権チェック
3	Service	DB::transaction()開始
4	Service	ActivityLog::lockForUpdate()->find($id)
5	Service	ended_at が既にセット済み → AlreadyStoppedException → 422
6	Service	update(['ended_at' => now(), 'auto_stopped' => false])
7	DB	activity_logs の ended_at を更新
8	Controller	停止後のログ（duration_minutes 計算済み）を JSON で返却（200）
9	Client	レスポンス受信次第、配賦モーダルを自動表示

例外系

- ▶️ **すでに停止済み：** 422 / message: 'この活動ログはすでに停止されています'
- ▶️ **他ユーザーのログ：** 403 Forbidden

5.3 配賦モーダル → 時間配賦

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	計測停止レスポンス受信後、配賦モーダルを表示
2	Client	タスク選択・分単位で時間入力。合計が総時間超過なら赤色警告表示
3	Client	「保存」押下 → POST /api/tasks/{id}/time-logs を送信（タスクごとに 1 リクエスト）
4	Middleware	Sanctum 認証 → TaskTimeLogPolicy で所有権チェック
5	Controller	StoreTaskTimeLogRequest でバリデーション（duration_minutes 必須・min:1）
6	Service	activity_log_id が指定された場合、ActivityLog::find()で ended_at が NULL か確認
7	Service	ended_at=NULL なら TrackingInProgressException → 422
8	Service	worked_on = activity_log.started_at の日付を自動セット
9	Model	TaskTimeLog::create([task_id, activity_log_id, duration_minutes, worked_on, note])
10	DB	task_time_logs に INSERT
11	Controller	作成した配賦レコードを JSON で返却（201）
12	Client	全タスクへの配賦完了後、モーダルを閉じる

例外系

- ▶️ **計測中ログへの配賦：** 422 / message: '計測中の活動ログには配賦できません'
- ▶️ **スキップ：** モーダルを閉じる。活動ログ一覧に「配賦する」ボタンが表示される

6. 例外系・バッチフロー

6.1 24 時間自動停止バッチ

Scheduler	Command	Service	DB	Client

Step	レイヤー	処理内容
1	Scheduler	毎時 0 分に AutoStopActivityLogs コマンドを起動（Kernel.php）
2	Command	ActivityLog::whereNull('ended_at')->where('started_at', '<=', now()->subHours(24))->get()
3	DB	該当レコードを取得（ended_at=NULL かつ開始から 24 時間以上経過）
4	Service	update(['ended_at' => started_at + 24h, 'auto_stopped' => true])
5	DB	activity_logs の ended_at・auto_stopped を更新
6	Client	次回ダッシュボードアクセス時に auto_stopped=true のログを検出

Step	レイヤー	処理内容
		して通知バナー表示
7	Client	ユーザーが通知を確認 → POST /api/activity-logs/{id}/acknowledge
8	DB	acknowledged_at に現在時刻をセット

※ バッチは毎時実行のため、最大1時間の遅延が発生する可能性がある（許容範囲とする）

6.2 手動時間入力（ログなし配賦）

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	タスク詳細サイドパネルの「時間を手動追加」ボタンを押下
2	Client	duration_minutes・worked_on・note を入力
3	Client	POST /api/tasks/{id}/time-logs（activity_log_id=null）を送信
4	Middleware	Sanctum 認証 → TaskTimeLogPolicy で所有権チェック
5	Controller	StoreTaskTimeLogRequest でバリデーション
6	Service	activity_log_id が NULL のため activity_log のチェックをスキップ
7	Model	TaskTimeLog::create([task_id, activity_log_id=null, duration_minutes, worked_on, note])
8	DB	task_time_logs に INSERT（activity_log_id=NULL）
9	Controller	作成した配賦レコードを JSON で返却（201）

6.3 ソフトデリートされたタスクの配賦表示

Client	Controller	Model	DB

Step	レイヤー	処理内容
1	Client	活動ログ一覧を表示
2	Controller	ActivityLog 一覧取得時に task_time_logs を with で eager ロード
3	Model	TaskTimeLog→Task（withTrashed()でソフトデリート済みも取得）
4	DB	deleted_at が NULL でないタスクも含めて結合
5	Controller	deleted_at がある場合は task_title を「削除済みタスク」として返却
6	Client	「削除済みタスク」をグレーテキストで表示・復元ボタンを表示

6.4 権限エラー（403）の共通フロー

Client	Middleware	Controller

Step	レイヤー	処理内容
1	Client	他ユーザーのリソースに対するリクエストを送信
2	Middleware	Policy クラスで user_id の一致チェック
3	Middleware	不一致 → AuthorizationException をスロー
4	Controller	Handler.php で 403 レスポンスに変換
5	Client	トースト通知「アクセス権限がありません」を表示（3 秒で自動消去）

6.5 未認証（401）の共通フロー

Client	Middleware

Step	レイヤー	処理内容
1	Client	有効でないトークン or トークンなしでリクエスト送信
2	Middleware	Sanctum が認証失敗を検出
3	Middleware	401 Unauthorized を返却
4	Client	React の axios インターセプターで 401 を検知
5	Client	ローカルのトークンを削除してログイン画面へリダイレクト

7. ダッシュボード集計フロー

7.1 週次・月次集計取得

Client	Middleware	Controller	Service	DB

Step	レイヤー	処理内容
1	Client	GET /api/activity-logs/summary?period=week を送信
2	Middleware	Sanctum 認証チェック
3	Service	period=week の場合、Carbon::now('Asia/Tokyo')->startOfWeek(Carbon::SUNDAY)で週の開始日（日曜）を算出
4	Service	期間を UTC に変換してクエリ条件を設定
5	DB	activity_logs: whereBetween('started_at', [$start, $end])で WHERE 絞り込み（開始日基準）

Step	レイヤー	処理内容
6	DB	TIMESTAMPDIFF(MINUTE, started_at, ended_at)で総活動時間を集計
7	DB	task_time_logs: task_id→project_id でプロジェクット別配賦時間を集計
8	DB	日別集計（GROUP BY DATE(started_at)）
9	Controller	集計結果を SummaryResource で整形して返却（200）
10	Client	ダッシュボードのウィジェットに表示

※ ended_at=NULL の計測中ログは集計から除外（WHERE ended_at IS NOT NULL）
※ 日跨ぎログはstarted_at の日付に全時間をカウント（開始日基準）

8. React フロントエンドフロー

8.1 打刻ボタンの状態管理

Client	API

Step	レイヤー	処理内容
1	Client	ページ読み込み時に GET /api/activity-logs で is_tracking=true のログを確認
2	Client	計測中のログがあれば「停止」ボタンを表示（タイマーも起動）
3	Client	計測中のログがなければ「開始」ボタンを表示
4	Client	「開始」押下 → POST /api/activity-logs/start
5	Client	201 レスポンス受信 → ボタンを「停止」に切替、タイマー開始
6	Client	「停止」押下 → POST /api/activity-logs/{id}/stop
7	Client	200 レスポンス受信 → ボタンを「開始」に切替、タイマー停止
8	Client	配賦モーダルを自動表示

8.2 タスク詳細サイドパネルの開閉

Client	API

Step	レイヤー	処理内容
1	Client	タスク行をクリック
2	Client	GET /api/tasks/{id} を送信
3	Client	GET /api/tasks/{id}/time-logs を送信（累計時間取得）
4	Client	レスポンス受信後、右側からサイドパネルをスライドイン表示

Step	レイヤー	処理内容
5	Client	タスク情報・サブタスク・累計時間を表示
6	Client	フィールドを編集 → PUT /api/tasks/{id} を送信
7	Client	200 レスポンス受信後、パネル内の表示を更新
8	Client	パネル外クリック or 閉じるボタンでスライドアウト

8.3 タスク横断一覧の URL クエリ同期

Client	API

Step	レイヤー	処理内容
1	Client	/tasks ページ読み込み時に URL のクエリパラメータを解析
2	Client	クエリパラメータからフィルター・ソート状態を React state に復元
3	Client	GET /api/tasks?{クエリパラメータ} を送信
4	Client	フィルター・ソートを変更 → React state を更新
5	Client	state の変更を検知して URL のクエリパラメータを更新（history.pushState）
6	Client	300ms デバウンス後に API 再取得
7	Client	ローディング中はスケルトン UI を表示
8	Client	レスポンス受信後にタスク一覧を更新

8.4 ローディング・エラー・空状態の制御

Client	API

Step	レイヤー	処理内容
1	Client	API リクエスト送信時に isLoading を true に設定
2	Client	isLoading=true の間はスケルトン UI を表示
3	Client	レスポンス受信 → isLoading を false に設定
4	Client	data.length === 0 の場合は空状態 UI を表示（「まだ〇〇がありません」＋作成ボタン）
5	Client	API エラー（4xx/5xx）の場合は axios インターセプターで捕捉
6	Client	401 の場合：トークン削除→ログイン画面リダイレクト
7	Client	403 の場合：トースト「アクセス権限がありません」（赤・3 秒）
8	Client	500 の場合：トースト「エラーが発生しました。再度お試しください」（赤・3 秒）

Step	レイヤー	処理内容
9	Client	作成・更新成功の場合：トースト「保存しました」（緑・3 秒）

9. フロー一覧

番号	フロー名	主な処理
2.1	ユーザー登録	bcrypt ハッシュ化・Sanctum トークン発行・user_settings 自動作成
2.2	ログイン	Auth::attempt()・トークン再発行
3.1	プロジェクット作成	デフォルトステータス 3 件自動生成
3.2	ステータスカスタマイズ	Policy 所有権チェック・名前のみ更新
4.1	タスク作成	階層 depth 計算・循環参照チェック
4.2	タスク削除・復元	SoftDeletes・サブタスク連動
5.1	計測開始	SELECT FOR UPDATE・同時計測防止
5.2	計測停止	SELECT FOR UPDATE・重複停止防止・配賦モーダル起動
5.3	時間配賦	計測中チェック・worked_on 自動セット・超過警告
6.1	24 時間自動停止バッチ	毎時 cron・auto_stopped=true・通知確認フロー
6.2	手動時間入力	activity_log_id=NULL でタスク時間を直接登録
6.3	削除済みタスクの表示	withTrashed()・「削除済みタスク」表示・復元ボタン
6.4	権限エラー	Policy→403→トースト通知
6.5	未認証エラー	401→axios インターセプター→ログイン画面リダイレクト
7.1	週次・月次集計	Carbon 期間算出・開始日基準・activity+task_time_logs の 2 軸集計
8.1	打刻ボタン状態管理	is_tracking フラグ・タイマー起動・配賦モーダル連携
8.2	サイドパネル開閉	タスク詳細・累計時間の並列取得・インライン編集
8.3	URL クエリ同期	フィルター状態を URL に反映・デバウンス 300ms
8.4	ローディング・エラー制御	スケルトン UI・トースト通知・空状態表示